

Three green apples are arranged in a cluster on a white background. One apple is in the foreground, slightly to the right, and two are behind it, one to the left and one to the right. The apples are bright green with some yellowing at the stems.

人工知能

IE229 – ARTIFICIAL INTELLIGENCE

第9回講義: PyTorch プログラミング (1)

Lecture 9: PyTorch Programming (1)

二宮 崇 (Takashi Ninomiya)

愛媛大学 (Ehime University)

ninomiya.takashi.mk@ehime-u.ac.jp

第9回はPyTorchプログラミングについて勉強します。

今回の講義内容

- **深層学習(ディープラーニング)演習**
 - PyTorchを使った画像認識演習



2

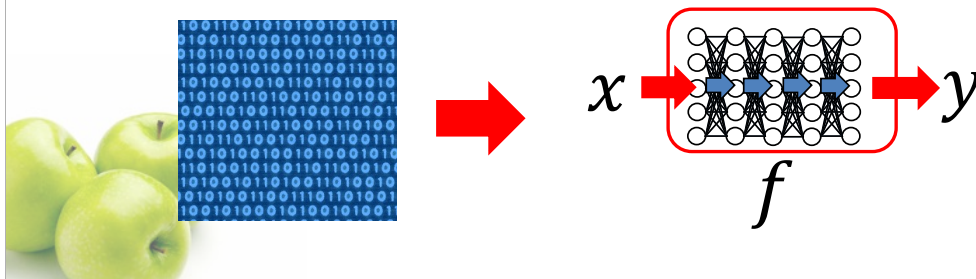
今回は深層学習ツールであるPyTorchの使い方を勉強します。

【復習】機械学習

- 機械学習 = データから関数を学習する学問分野や技術



- 深層学習も機械学習の一種

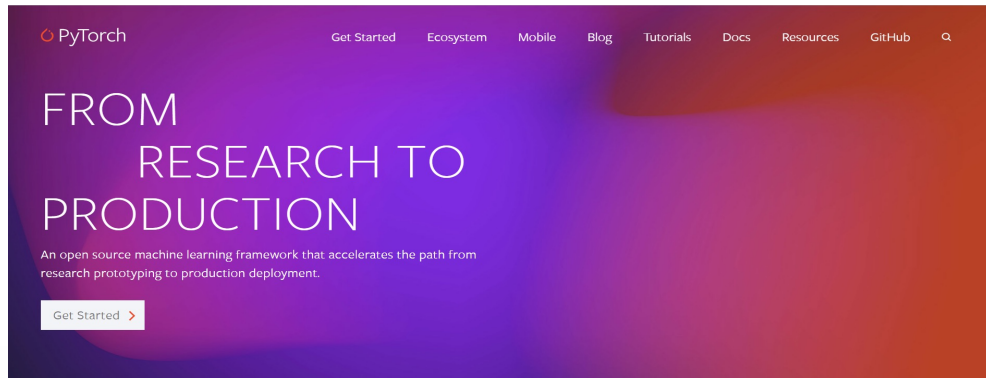


3

機械学習は前回にも説明したように、データから関数を学習する学問分野や技術のことです。現在の人工知能技術の中核にあたる深層学習も実は機械学習の一種です。一見、人工知能と関数の学習は関係ないように思いますが、深層学習で学習するネットワークは実は一つの大きな関数になっていて、実際、深層学習のモデルは数式で表されることが多いです。

PyTorch

- <https://pytorch.org/>



4

この授業ではPyTorchと呼ばれるFacbookのAI Research Lab (FAIR)が開発した深層学習ツールについて勉強します。現在よく用いられている深層学習ツールとしては、PyTorch, TensorFlow, Kerasの3つが有名ですが、KerasはTensorFlowをバックエンドとして用いているので、事実上、PyTorchかTensorFlowの二択と言って良いでしょう。この講義ではそのうちの一つのPyTorchについて学びます。

PyTorch

- Facebookによって開発されたニューラルネットワーク学習フレームワーク
 - Define-by-Runによるネットワーク定義
 - データごとに構成が変化するネットワークにも柔軟に対応
 - ⇔ Define-and-Run
 - Pythonさえ知っていれば記述できる



5

PyTorchはFacebook AI Research Lab (FAIR)によって開発されたニューラルネットワーク学習フレームワークで、大きな特徴としては、Define-by-Runによるネットワーク定義があります。Define-by-Runは、データごとに構成が変化するネットワークにも柔軟に対応できるネットワーク定義の方法になります。日本発のChainerという深層学習ツールがあるのですが、Define-by-RunというアイデアはこのChainerで初めて発明されて実装されています。PyTorchはこのChainerのDefine-by-Runというアイデアを取り込み、かつ非常に高速に動作することから高い人気を得ています。(Chainerの開発元のPFNは、2019年12月にChainerの開発を中止して、PyTorchに移行しました)

PyTorchはPythonさえ知っていれば深層学習のプログラムが書けるため、非常に扱いやすい深層学習ツールとなっています。

PyTorch

- **Define-and-Run (Caffe, TensorFlow)**

- ネットワークを先につくってから、データをネットワークに流し込む
- 一度作ったネットワークを変更することができない
- 計算が速い

- **Define-by-Run (PyTorch, Chainer)**

- データを流し込むと同時にネットワークを作る
- データ入力時にデータにあわせてネットワークを作ることができる
 - 文のように可変サイズ of データを扱うときに便利
 - データの入力ごとにネットワークを作っているため遅い



6

Define-by-Runと対立するネットワークの構築方法として、Define-and-Runというのがあります。

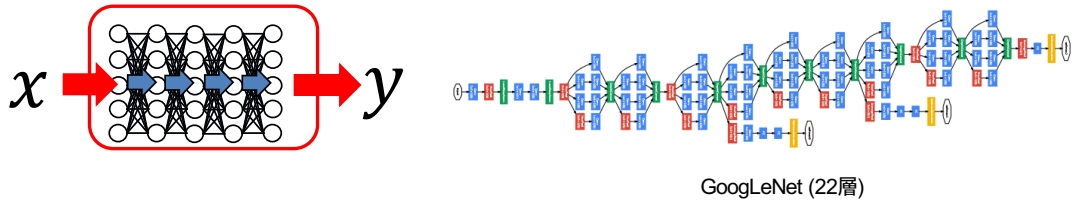
Define-and-RunはCaffeやTensorFlowで採用されている方法なのですが、ネットワークを先に作ってから、データをネットワークに流し込む、というやり方で学習と推論を行います。一度作ったネットワークを変更することができない、という欠点がありますが、ネットワークを使い回すことができるため、計算が速いという利点があります。

Define-by-RunはPyTorchやChainerで採用されている方法なのですが、データを流し込むと同時にネットワークを作ることを行っています。データ入力時にデータにあわせてネットワークを作ることができるので、文のように可変サイズのデータを扱うときに便利という利点があります。一方、データの入力ごとにネットワークを作っているために遅い、という欠点があります。

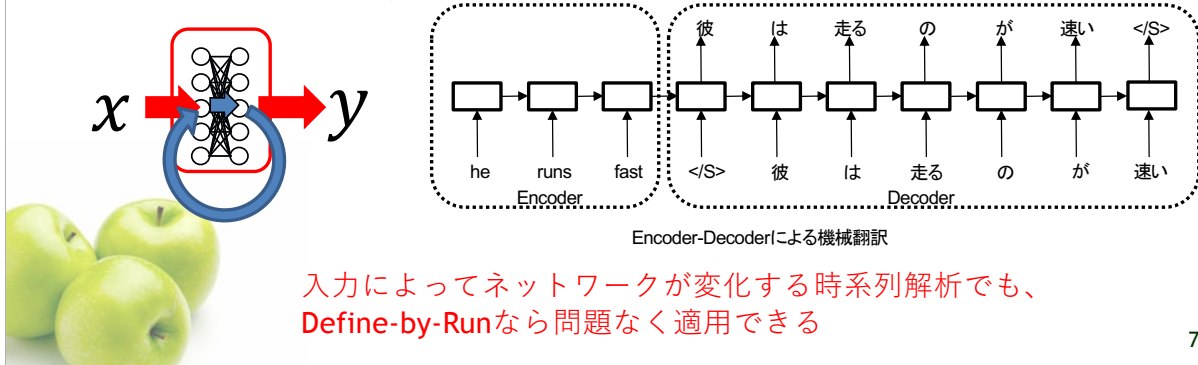
これらの特徴から、企業での産業応用にはTensorFlowが主に使われて、大学や企業での研究用にはPyTorchが主に使われている、という状況です。

Define-by-Runの利点

- フィードフォワードニューラルネットワーク



- リカレントニューラルネットワーク



7

時系列データを扱うリカレントニューラルネットワークではこのスライドの図にあるように入力によってそのネットワークの形が変化しますが、Define-by-Runなら問題なくこれらのネットワークに対応することができます。

Google ColaboratoryでPyTorchを動かす

- まず、import torchが問題なく動くか確認しよう

```
[1] import torch
```



実行

```
[1] import torch
```



(参考)

Ctrl+Enter: セルの実行

(参考)

Alt+Enter: セルの実行+コードの追加



このようにエラーが特にでていなければok!

8

それではGoogle ColaboratoryでPyTorchを動かしてみましょう。

ColaboratoryにはすでにPyTorchがインストールされているので、importを実行するだけで利用可能となります。まず、「import torch」を実行して何もエラーがでないことを確認しましょう。

テンソルを作る

- **torch.zeros**: 0で埋まったテンソル(多次元行列)を作る
- **torch.ones**: 1で埋まったテンソルを作る
- **torch.tensor**: リストで表された行列からテンソルを作る
 - **requires_grad=True**をつけることで勾配を計算してくれる

```
[1] import torch

a = torch.zeros((10, 15), requires_grad=True)
print(a)
b = torch.ones((10, 15), requires_grad=True)
print(b)
c = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
print(c)
```



うまくいけば10x15の0の行列
と10x15の1の行列と[1,2,3]の
行列が表示される

実行結果

```
tensor([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
requires_grad=True
tensor([[1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
requires_grad=True
tensor([1., 2., 3.], requires_grad=True)
```

9

PyTorchの基本はテンソル型というデータ構造を作成することです。テンソル型は、NumPyのArray型とほぼ同じ機能を持つスカラー、ベクトル、行列、テンソルを表現するためのデータ型です。Array型との違いは誤差逆伝播法を使って微分の計算ができることと、GPUが使えることです。

テンソル型のデータはtorch.tensorという命令で作ることができます。その他にtorch.zerosで0が埋まったテンソルを作ることができ、torch.onesで1が埋まったテンソルを作ることができます。torch.tensorはリストで表された行列からテンソルを作ることができます。また、誤差逆伝播法を使って勾配を計算するには、requires_grad=Trueというオプションをつけてあげる必要があります。

このスライドのプログラムをcolaboratoryで実行してみましょう。

ネットワークの作成と計算

- 次の関数 f の $f(1,2,3)$ と $\nabla f(1,2,3)$ の計算を試みよう

$$f(x_1, x_2, x_3) = (x_1 - 2x_2 - 1)^2 + (x_2x_3 - 1)^2 + 1$$

- 解析的な答え

$$f(1,2,3) = 42$$

$$\frac{\partial f}{\partial x_1} = 2(x_1 - 2x_2 - 1)$$

$$\frac{\partial f}{\partial x_2} = -4(x_1 - 2x_2 - 1) + 2(x_2x_3 - 1)x_3$$

$$\frac{\partial f}{\partial x_3} = 2(x_2x_3 - 1)x_2$$

$$\nabla f(1,2,3) = \left(\frac{\partial f}{\partial x_1}(1,2,3), \frac{\partial f}{\partial x_2}(1,2,3), \frac{\partial f}{\partial x_3}(1,2,3) \right) = (-8, 46, 20)$$

10

続いて、ネットワークの作成と勾配の計算を行ってみます。例として、

$$f(x_1, x_2, x_3) = (x_1 - 2x_2 - 1)^2 + (x_2x_3 - 1)^2 + 1$$

の $f(1,2,3)$ と $\nabla f(1,2,3)$ を計算してみます。

この解析的な答えは、まず、 $f(1,2,3) = 42$ で、勾配は、

$$\frac{\partial f}{\partial x_1} = 2(x_1 - 2x_2 - 1)$$

$$\frac{\partial f}{\partial x_2} = -4(x_1 - 2x_2 - 1) + 2(x_2x_3 - 1)x_3$$

$$\frac{\partial f}{\partial x_3} = 2(x_2x_3 - 1)x_2$$

となるため、

$$\nabla f(1,2,3) = \left(\frac{\partial f}{\partial x_1}(1,2,3), \frac{\partial f}{\partial x_2}(1,2,3), \frac{\partial f}{\partial x_3}(1,2,3) \right) = (-8, 46, 20)$$

となります。

関数と勾配の計算

- 次の関数 f の $f(1,2,3)$ と $\nabla f(1,2,3)$ の計算を試みよう

$$f(x_1, x_2, x_3) = (x_1 - 2x_2 - 1)^2 + (x_2x_3 - 1)^2 + 1$$

```
x1 = torch.tensor(1.0, requires_grad=True)
x2 = torch.tensor(2.0, requires_grad=True)
x3 = torch.tensor(3.0, requires_grad=True)
```

$(x_1, x_2, x_3) = (1, 2, 3)$

```
z = (x1 - 2*x2 - 1)**2 + (x2*x3 - 1)**2 + 1
```

$f(1, 2, 3)$ の計算
計算結果をzに代入

```
print("z: ", z)
print("z: ", z.item())
z.backward()
```

zでテンソル型の結果が表示される。
z.item()とすることでzの結果が実数型
で得られる

```
print("x1: ", x1.grad)
print("x2: ", x2.grad)
print("x3: ", x3.grad)
```

z.backward()でzに対する勾配の計算

x.gradで、zをxで偏微分した結果が得られる

11

PyTorchは誤差逆伝播法を用いて勾配を自動的に計算する機能がありますので、この機能を用いて先程の例の勾配を計算してみましょう。

プログラムは図のようになります。

```
x1 = torch.tensor(1.0, requires_grad=True)
x2 = torch.tensor(2.0, requires_grad=True)
x3 = torch.tensor(3.0, requires_grad=True)
```

でx1に1、x2に2、x3に3をいれて、かつ、この変数に対し、勾配を計算するようrequires_grad=Trueをつけておきます。

```
z = (x1 - 2*x2 - 1)**2 + (x2*x3 - 1)**2 + 1
```

で関数 f の計算を行っていますが、実はこの式はただの計算式ではなくて、x1やx2、x3がテンソル型であるため、これらに各種計算を行うと計算グラフを作りながら関数 f の計算を行います。x1~x3が計算グラフにおける変数ノードになっていて、これらに対して各種計算を行うとその都度新しい変数ノードが作成されます。このようにして計算グラフを作ると同時に関数の計算を行って、最終的にzの値をテンソル型として得ます。つまり、順伝播の計算をこの1行で行っていることになります。

zをprintすることでzの値をテンソル型としてみることができます。実数型として利用したい場合は、z.item()を実行することで実数型を得ることができます。

続いて、

```
z.backward()
```

を実行すると、zを起点とする逆伝播の計算を実行します。つまり、この実行が終わった時点で、各入力変数にはzに対する微分値が入っていることになります。

その微分値の計算結果は、各入力変数xに対して、x.gradの中に入っています。x.gradはテンソル型ですが、実数型が欲しい場合は、x.grad.item()を実行すると実数型が得られます。

関数と勾配の計算

- 計算結果

```
z: tensor(42., grad_fn=<AddBackward0>)  
z: 42.0  
x1: tensor(-8.)  
x2: tensor(46.)  
x3: tensor(20.)
```

解析的な計算結果と一致している

- 解析的な計算結果(2ページ前のスライド)

$$f(1,2,3) = 42$$
$$\nabla f(1,2,3) = \left(\frac{\partial f}{\partial x_1}(1,2,3), \frac{\partial f}{\partial x_2}(1,2,3), \frac{\partial f}{\partial x_3}(1,2,3) \right) = (-8, 46, 20)$$



PyTorchにはこのように関数と勾配の計算を自動的に行う機能がある

12

この実行結果はこのようになります。テンソル型のzはその中に42という結果が入っていて、なんらかの関数がついていることがわかります。z.item()を実行した結果は、実数型の42という結果が返ってきていることがわかります。

続いて、微分値の方ですが、解析的な計算結果と一致していることがわかると思います。

このようにPyTorchには関数と勾配の計算を自動的に行う機能があることがわかります。誤差逆伝播法を用いているので、ニューラルネットワークのような巨大な関数だったとしても、この機能を用いて、関数計算と勾配を計算することができます。あとは、関数と損失関数を組み合わせて、データ全体の損失関数の勾配を求めれば、学習ができるということになります。

PyTorchを使った勾配降下法による最小二乗法

```
import numpy as np
import matplotlib.pyplot as plt
import torch
```

```
lr = 0.01
X = [1,2,4,5]
Y = [2,3,3,4]
a = 0.0
b = 0.0
```

パラメータ__a, __bの設定
損失lossの初期化

```
for e in range(300):
    __a = torch.tensor(a, requires_grad=True)
    __b = torch.tensor(b, requires_grad=True)
    loss = torch.tensor(0.0)
    for i in range(len(X)):
        loss = loss + (Y[i] - __a*X[i]-__b)**2
    loss.backward()
    a = a - lr * __a.grad.item()
    b = b - lr * __b.grad.item()
    print("epoch: ", e, "a: ", a, "b: ", b, "loss: ", loss.item())
```

各データの損失の累積を計算

損失に対する勾配を自動計算

勾配を使ってパラメータ更新

```
def f(x):
    return a*x+b

plt.scatter(X, Y, color="red", marker=".")
pltX = np.linspace(0, 6, 100)
pltY = f(pltX)
plt.plot(pltX, pltY)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Pythonプログラミングで求めた回帰モデルをPyTorchで計算してみよう

$D = \{(1,2), (2,3), (4,3), (5,4)\}$
 $f(x) = ax + b$

Pythonプログラミングで求めた回帰モデルをPyTorchで計算してみよう

$D = \{(1,2), (2,3), (4,3), (5,4)\}, \quad f(x) = ax + b$

に対して、aとbを求めます。

lr = 0.01

X = [1,2,4,5]

Y = [2,3,3,4]

a = 0.0

b = 0.0

は初期設定です。

for e in range(300):

で300回のエポックを回します。

__a = torch.tensor(a, requires_grad=True)

__b = torch.tensor(b, requires_grad=True)

でまず、計算グラフの変数ノードを作ります。値は現在のaとbの値として、変数ノード__aと__bを作ります。続いて、損失(誤差)を

loss = torch.tensor(0.0)

で用意します。各データの損失をこのlossに足し込んでいって最終的に残るlossが計算グラフ全体の出力になります。

for i in range(len(X)):

loss = loss + (Y[i] - __a*X[i]-__b)**2

各データの損失は正解とモデルによる予測値(__a*X[i] + __b)との差分の二乗になります。

loss.backward()

で、誤差逆伝播法を適用して、全データの損失に対する__aと__bの勾配を計算します。

13

```
a = a - lr * __a.grad.item()
b = b - lr * __b.grad.item()
```

によって、求めた勾配に学習率 $\times -1$ を掛けたものをa,bに足し込みます。

その下のプログラムを入力すると学習された直線が可視化されますので、どんな結果になっているか見たい人は入力してみてください。

PYTORCHを使った深層学習

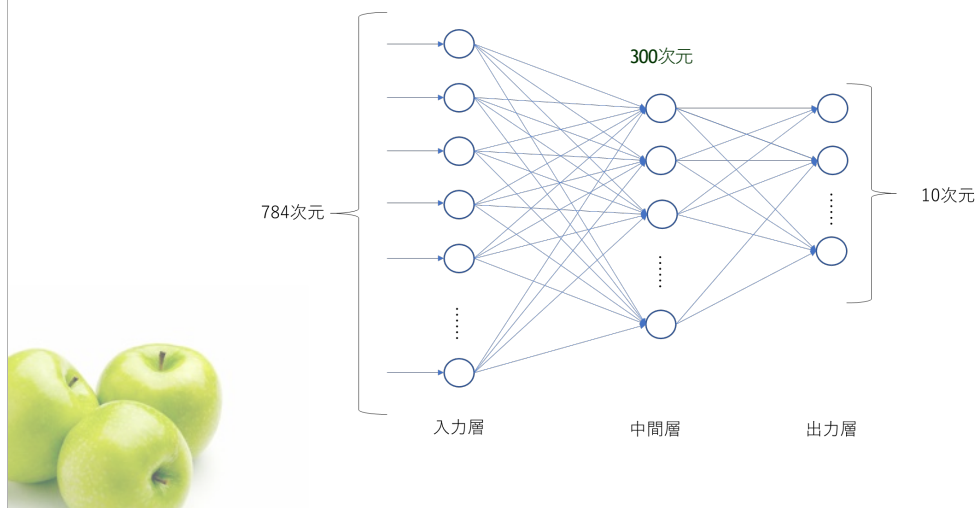


14

次は、PyTorchを使った深層学習のプログラムを書いてみます。

多層パーセプトロン

- 入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワークを作って白黒画像(28x28)の画像認識を行います

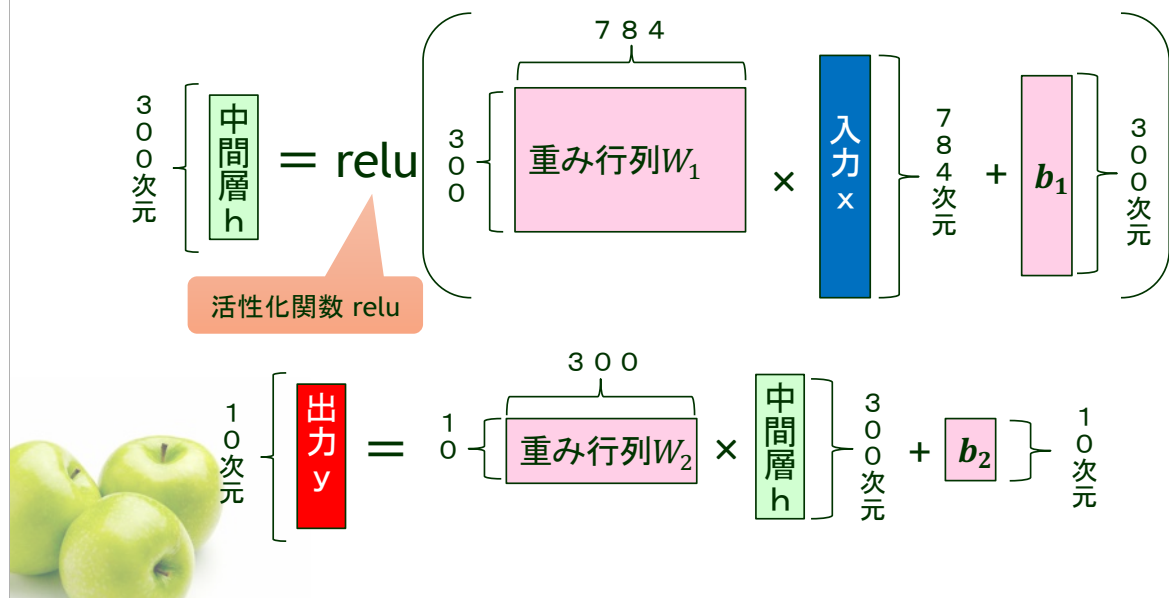


15

今回は、入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワークを作って白黒画像(28x28)の画像認識を行います。

多層パーセプトロン

- 入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワークの計算



16

入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワークの計算は具体的には図のように行われます。

784次元の入力に対して、300x784の重み行列を掛けて、300次元のバイアス項を足して、ReLUをかけると、300次元の中間層 h が得られます。

続いて、その中間層 h に対して、10x300の重み行列を掛けて、10次元のバイアス項を足して、10次元の出力を得ます。

線形変換(全結合層) torch.nn.Linear

- torch.nn.Linear

- 線形変換(全結合層)のモジュール
 - $y = Wx + b$ を計算する
 - W (行列)と b (ベクトル)がパラメータ
- torch.nn.Linear(in_dim, out_dim)
 - in_dim次元のベクトルからout_dim次元のベクトルに射影する線形変換を計算する層を作る
- 例: torch.nn.Linear(3, 4)
 - 3次元ベクトルから4次元ベクトルに射影する線形変換



17

線形変換(全結合層のこと)を行うモジュールがtorch.nn.Linearです。このモジュールを用いることで、 $y = Wx + b$ を計算します。 W (行列)と b (ベクトル)がパラメータとなります。

torch.nn.Linear(in_dim, out_dim)と書くことで、in_dim次元のベクトルからout_dim次元のベクトルに射影する線形変換を計算する層を作ります。例えば、torch.nn.Linear(3, 4)と書くと、3次元ベクトルから4次元ベクトルに射影する線形変換を作れます。

活性化関数 ReLU (Rectified Linear Unit) 関数

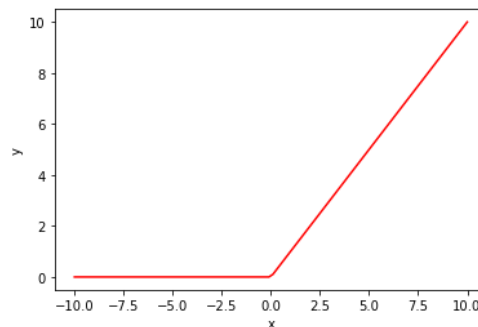
- ReLU関数

- 出力の活性化(オン・オフ)を表す非線形関数

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

- `torch.nn.functional.relu(x)`

- `x`に対して、 $h(x)$ の計算をする



18

ReLUは、`torch.nn.functional.relu(x)`と書くことでReLU関数を呼び出すことができます。`x`に対して、

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

の計算を行います。

データ MNIST

- MNIST

- 手書き文字の認識を行うタスク
- 画像に対して、どの数字が書かれているかを予測する。



- 訓練データは60,000データ
- テストデータは10,000データ
- 各データは画像とラベルのペア
- 各画像は28×28ピクセル
- ラベルは0～9までのいずれかの数字。画像に対する正解の数字。



19

データはMNISTと呼ばれるデータを用います。このデータは手書き文字の認識を行うタスクのためのデータです。画像に対して、どの数字が書かれているか予測します。

訓練データは60,000データ、テストデータは10,000データあります。各データは画像とラベルのペアになっていて、各画像は28x28ピクセルのグレースケールの画像です。ラベルは0から9までのいずれかの数字で、画像に対する正解の数字が与えられています。

データの読み込み

● データ読み込みのためのプログラム

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F
import torchvision as tv
```

ライブラリの読み込み

```
train_dataset = tv.datasets.MNIST(root=".", train=True,
                                   transform=tv.transforms.ToTensor(),
                                   download=True)
test_dataset = tv.datasets.MNIST(root=".", train=False,
                                   transform=tv.transforms.ToTensor(),
                                   download=True)
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                             batch_size=100,
                                             shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
                                           batch_size=100,
                                           shuffle=False)
```

訓練データとテストデータの読み込み
(初めて実行するときはデータをネットからダウンロードする)

訓練データとテストデータのミニバッチ処理
・ミニバッチサイズ=100
・データの順番をシャッフル

20

まず、データを読み込むためのプログラムを書きましょう。最初に、
`import numpy as np`
`import matplotlib.pyplot as plt`
`import torch`
`import torch.nn.functional as F`
`import torchvision as tv`
と書くことでライブラリを読み込みます。上3行は今までのものと同じです。新しく
`import torch.nn.functional as F`
`import torchvision as tv`
が追加されています。上側は`torch.nn.functional.*`と書くところを`F.*`と省略して書けるようになるための宣言です。下側は`torchvision`というライブラリを読み込んでいます。

`Torchvision`は有名な機械学習用のデータセット、モデル、画像認識のための画像変形のためのデータやモジュールを集めたライブラリです。

`tv.datasets.MNIST`でMNISTのデータをロードすることができます。

```
train_dataset = tv.datasets.MNIST(root=".", train=True,
                                   transform=tv.transforms.ToTensor(),
                                   download=True)
```

でMNISTの訓練データ(学習用データ)をインターネット上のサーバーからダウンロードすることができます。一度読み込めばドライブ上に保存されるので、再度サーバーにアクセスしないようになっています。

```
test_dataset = tv.datasets.MNIST(root=".", train=False,
                                   transform=tv.transforms.ToTensor(),
                                   download=True)
```

によって、MNISTのテストデータをダウンロードすることができます。訓練データとテ

ストデータの読み込みの違いはtrain=True/Falseで切り分けられています。
Transform=tv.transforms.ToTensor()でデータを読み込む際にテンソル型に変換して読み込みます。

続いて、

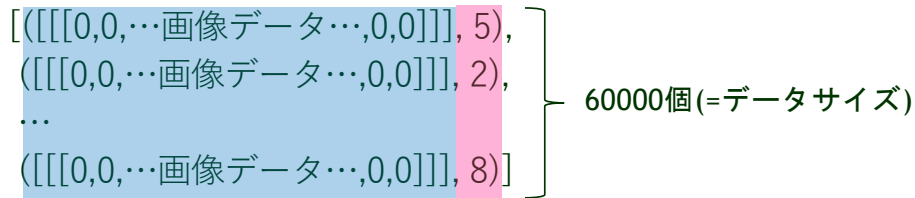
```
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,  
                                             batch_size=100,  
                                             shuffle=True)  
test_loader = torch.utils.data.DataLoader(dataset=test_dataset,  
                                           batch_size=100,  
                                           shuffle=False)
```

によって、訓練データとテストデータを読み込むためのローダーを設定します。順にデータを読めば良いだけなのですが、大きなデータの場合はミニバッチという単位のデータごとに勾配法を適用することを行います。そのためにデータをbatch_sizeごとに切り分けて自動的に読み込めるようにしてくれるモジュールがDataLoaderです。Torch.utils.data.DataLoaderで利用することができます。batch_sizeの指定の他に、データの順番をシャッフルするかどうか指定することができます。学習時には様々な良い学習モデルを得るために初期値を変えたり、ノイズを入れたり、データの順番をシャッフルすることがあります。たまたま悪いモデルやたまたま良いモデルを避けることで、汎化性能が向上します。そのため、学習時にはシャッフルをして、テストの際は順番を入れ替えることに意味はないので、shuffle=Falseとします。

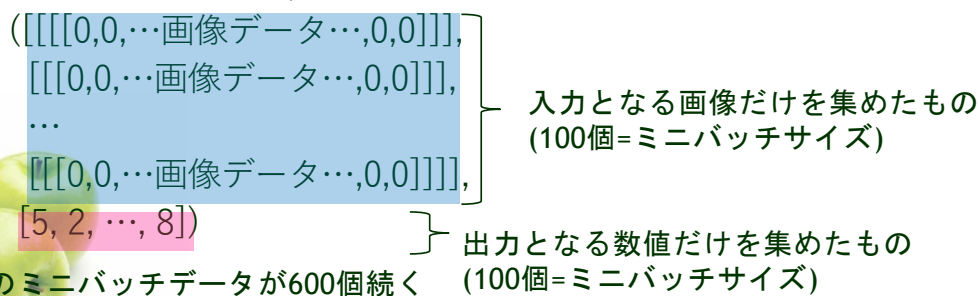
DataLoaderがしていること

● ミニバッチ形式への変換

- もともとのデータ (1×28×28のリストと数値のペアのリスト)


[[[[0,0,...画像データ...,0,0]], 5),
[[[[0,0,...画像データ...,0,0]], 2),
...
[[[[0,0,...画像データ...,0,0]], 8]] } 60000個 (=データサイズ)

- 変換後のデータ (1×28×28のリストと数値のリストのペア)


[[[[0,0,...画像データ...,0,0]],
[[[0,0,...画像データ...,0,0]],
...
[[[0,0,...画像データ...,0,0]]],
[5, 2, ..., 8]] } 入力となる画像だけを集めたもの
(100個=ミニバッチサイズ)
出力となる数値だけを集めたもの
(100個=ミニバッチサイズ)

100個のミニバッチデータが600個続く

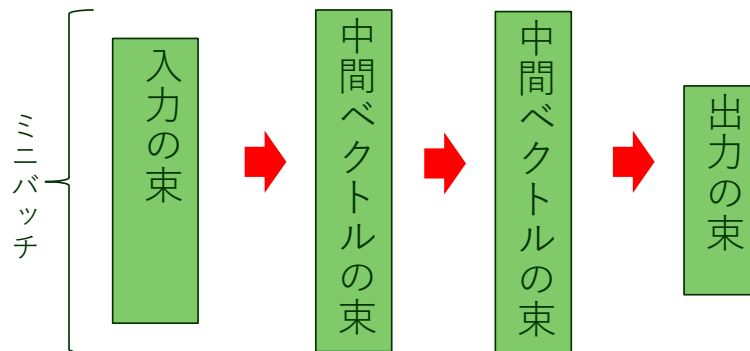
21

DataLoaderはミニバッチ形式に変換してくれることが大きな役割となりますが、ミニバッチの変換は図のように行われます。

元々のデータは1×28×28のリスト(画像)と数値(正解ラベル)のペアのリストになっているので、図の上側のように並んでいます。このデータを変換後は、1×28×28のリストと数値のリストのペアに変更します。つまり、水色のところが画像データで、ピンク色のところがラベルデータになるのですが、画像データをすべてまとめて、その後に、ラベルデータを全てまとめてペアにしています。何故こんなことをするのかというと計算効率が関係してくるからです。

ミニバッチ

- ミニバッチ内のデータをまとめて左から右に流す(計算する)



- まとめて計算できる行列演算が増える
 - 行列演算が得意なGPUにとって非常に効率の良い計算方法



22

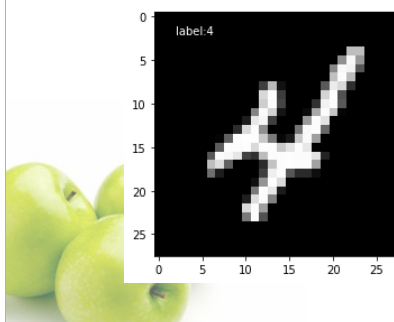
特にGPUを使ったときはたくさんのデータをまとめて渡してまとめて計算した方が非常に効率的になります。GPUはCPUと違って、非常にたくさんのコアからできているので(CPUは数十コアで、GPUは数千コア)、行列やベクトルの並列計算を非常に効率的に高速に計算できます。つまり、入力の束をまとめて与えて、まとめて中間ベクトルの束を計算し、まとめて出力の束を計算するようにすれば、並列計算効果が高く、非常に効率的になります。そのため、データを一つ一つ入力と出力のペアにするのではなく、入力の束と出力の束として与えるようにしているということです(前スライド参照)。

どんな画像が入っているのかみてみよう

- 先程のプログラムの下に次のプログラムを追加して実行してみよう

```
for i in range(10):  
    print(train_dataset[i])  
    plt.imshow(train_dataset[i][0][0], cmap="gray")  
    txt = "label:" + str(train_dataset[i][1])  
    plt.text(2, 2, txt, color="white")  
    plt.show()
```

train_dataset[i][0][0]でi番目の画像(28x28の白黒)
train_dataset[i][1]でi番目の正解ラベル



- ・ 具体的なデータの中身が表示される
- ・ このような画像が10枚表示される(左上の「label: 4」はこの画像の正解ラベル)

(参考) カラー画像だったら、
train_dataset[i][0][0] 赤(R)の画像
train_dataset[i][0][1] 緑(G)の画像
train_dataset[i][0][2] 青(B)の画像

今回は白黒なので、
train_dataset[i][0][0]
だけ使う

23

先程のプログラムの下に次のプログラムを追加して実行してみましょう。
train_datasetの先頭の10個のデータの具体的な中身(画像のテンソル型表示)と、
画像が表示されます。train_dataset[i][0][0]でi番目の画像(28x28の白黒)が手に入
ります。train_dataset[i][1]で正解ラベルが手に入ります。つまり、train_dataset[i][0]
が入力データで、train_dataset[i][1]で出力データが手に入るといことです。カラー
画像だったら
train_dataset[i][0][0] 赤(R)の画像
train_dataset[i][0][1] 緑(G)の画像
train_dataset[i][0][2] 青(B)の画像
が手に入るようになっているのですが、今回は白黒なので、train_dataset[i][0][0]だ
け使うので、最後に[0]が余計に一つ付くようになっています。

コメント化

- せっかく書いたプログラムを消すのはもったいないので、コメント化しておこう
 - 「#」をいれるとそこから行末までは無視される(コメント化)
 - コメント化したいプログラム部分を範囲選択して、Ctrl+/を入力すると、まとめてコメント化できる(もう一度同じことをするとコメント解除)

```
for i in range(10):  
    print(train_dataset[i])  
    plt.imshow(train_dataset[i][0][0], cmap="gray")  
    txt = "label:" + str(train_dataset[i][1])  
    plt.text(2, 2, txt, color="white")  
    plt.show()
```



```
# for i in range(10):  
#     print(train_dataset[i])  
#     plt.imshow(train_dataset[i][0][0], cmap="gray")  
#     txt = "label:" + str(train_dataset[i][1])  
#     plt.text(2, 2, txt, color="white")  
#     plt.show()
```

コメント化された部分は実行されない

プログラムをみやすくするために積極的にコメントをいれよう

(例) #初期設定
 #モデルの定義
 #ここから訓練

24

先程書いたプログラムですが、せっかく書いたので、コメント化して残しておきましょう。Pythonでは#をいれるとそこから行末まで無視されます。コメント化と言います。コメント化したいプログラム部分を範囲選択して、Ctrl+/を入力すると、まとめてコメント化できます。もう一度Ctrl+/を入力するとコメント解除になります。コメント化された部分は実行されないなので、プログラムを見やすくするために積極的にコメントをいれていきましょう。

ネットワークの作成

● ネットワークの設定とパラメータ最適化の設定

```
l1 = torch.nn.Linear(784, 300)
l2 = torch.nn.Linear(300, 10)
params = list(l1.parameters()) + list(l2.parameters())
optimizer = torch.optim.Adam(params)

def mynet(x):
    h = F.relu(l1(x))
    y = l2(h)
    return y
```

l1とl2は線形変換の関数

l1/l2.parameters()でl1とl2に含まれるパラメータを取り出す

Adamというパラメータ最適化手法を使う。パラメータ更新に関わるパラメータ群paramsを渡す

ネットワーク全体の計算を行う関数 mynet
・xが入力、yが出力
・hはニューラルネットワーク前半(784次元→300次元への線形変換l1と活性化関数ReLUの適用)
・yはニューラルネットワーク後半(hに対して300次元→10次元への線形変換l2を適用)

25

続いてネットワークの設定とパラメータ最適化の設定を行います。

線形変換の関数はtorch.nn.Linearで手に入りますので、

```
l1 = torch.nn.Linear(784, 300)
```

```
l2 = torch.nn.Linear(300, 10)
```

と書くことで、l1とl2に線形変換(全結合層)の関数が定義されます。

l1とl2に付随するパラメータは、l1.parameters()およびl2.parameters()で手に入ります。これをリスト形式にして結合することで全パラメータのリストにしておきます。

torch.optim.Adam(params)で、これらの全パラメータを保持したoptimizerを作成します。torch.optim.Adamを呼び出すことで、Adamと呼ばれるパラメータ最適化手法を使うことができます。このときにparamsにいった全パラメータを渡したAdamが作成されます。optimizerは学習時に渡されたパラメータを更新することを行います。

続いて、

```
def mynet(x):
```

```
    h = F.relu(l1(x))
```

```
    y = l2(h)
```

```
    return y
```

と定義することで、計算グラフを作るため(順方向の計算を行うため)の関数を作ります。mynetに入力xを渡して、

```
    h = F.relu(l1(x))
```

を計算し、中間層の結果をhにいて、

```
    y = l2(h)
```

とすることで、出力yを得ます。最後に出力yを返します。

ネットワークの学習

● 訓練データからの学習

データ全体を
10回学習する

train_loaderから100個ずつデータを受け取る
Imagesは画像データ100枚
labelsは正解ラベル100個

#学習

```
for e in range(10):
```

```
    loss = 0
```

```
    for images, labels in train_loader:
```

```
        images = images.view(-1, 28*28)
```

```
        optimizer.zero_grad()
```

```
        y = mynet(images)
```

```
        batchloss = F.cross_entropy(y, labels)
```

```
        batchloss.backward()
```

```
        optimizer.step()
```

```
        loss = loss + batchloss.item()
```

```
    print("epoch:", e, "loss:", loss)
```

(100×1×28×28)から(100×784)に変形

勾配を初期化

ネットワークの計算(ラベルの予測)

正解ラベルに対する出力の損失

損失に対する勾配を計算

パラメータ更新

ミニバッチでの損失をlossに足す



26

続いて、学習部分です。まず、学習の繰り返し回数(エポック数)ですが、ここでは、`for e in range(10):`

として、10回繰り返すようにしています。ミニバッチを繰り返し最適化するので、データ全体の損失を計上するために`loss`という変数を用意して初期化しておきます(`loss=0`)。

続いて、

`for images, labels in train_loader:`

によって、`train_loader`から画像の束を`image`に、ラベルの束を`labels`に代入します。この`images`と`labels`は100個分のデータとなっていて、これがミニバッチと呼ばれる単位になります。このミニバッチに対して最適化を行うことを繰り返します。

画像は、(100×1×28×28)のサイズのテンソルになっているので、これを(100×784)のテンソルに変形します。

```
images = images.view(-1, 28*28)
```

を使うと、テンソルの整形ができるのですが、`28*28=784`の単位で、リスト化してくれます。`Images.view(100, 28*28)`としても同じことになりますが、100の部分进行計算するのが面倒な場合や、変化する場合、-1を指定しておくで全体のサイズから自動的に計算してくれます。(詳しく言うと、`view`は`images.view(v,w,x,y,z)`に対し、`images`を一度フラットなリストに変換して、それを`v, w, x, y, z`サイズのテンソルに順番に詰め込むを行っています。)

勾配は逆伝播の際に足し込む作業を行いますので、勾配をまず初期化しておく必要があります。(そうでないと、前回の勾配計算の結果がそのまま残っているので、正しい勾配が計算できません)。`optimizer.zero_grad()`で格納されている全パラメータの勾配を初期化します(0にします)。

`y=mynet(images)`で画像の束をまとめてネットワークに渡して計算を行います。出力の束`y`と正解ラベルの束`labels`の交差エントロピーを`F.cross_entropy(y, labels)`で

計算します。これをbatchlossに格納します。batchlossに対して、`batchloss.backward()`を実行して、各パラメータの勾配を得ます。

この段階で各パラメータには勾配が計算されているはずなので、`optimizer.step()`を実行すると、optimizerが持っている全パラメータの更新をアルゴリズム(この場合はAdam)に従って行います。

```
loss = loss + batchloss.item()
```

で、ミニバッチ毎の損失を足していって、全データの損失をlossに格納するようにしておきます。これは学習がどれぐらいすすんでいるかみるために計算しているものです。全データに対するパラメータ更新がおわったあと、

```
print("epoch:", e, "loss:", loss)
```

と表示させることで、何エポックで、どれぐらいの損失が残っているのかわかるようになります。

ネットワークによる推論

● テストデータに対して学習したネットワークを評価する

```
#テスト
correct = 0
total = len(test_loader.dataset)
for images, labels in test_loader:
    images = images.view(-1, 28*28)
    y = mynet(images)
    pred_labels = y.max(dim=1)[1]
    correct = correct + (pred_labels == labels).sum()

print("correct:", correct.item())
print("total:", total)
print("accuracy:", correct.item()/total)
```

test_loaderから100個ずつデータを受け取る
Imagesは画像データ100枚
labelsは正解ラベル100個

(100×1×28×28)から(100×784)に変形

ネットワークの計算

ラベルの予測(最大値となるラベル)

100個のうち正解数がい
くつか数える

correctは正解数
totalはテストデータのデータ数
accuracyは精度

学習が終わったあと、テストデータを用いて、性能評価を行います。correctは正解数を格納するための変数です。0で初期化しておきます。

correct = 0

続いて、totalに全データ数を格納しておきます。これはテストセットの大きさをlen関数で評価すればokです。

total = len(test_loader.dataset)

あとは、テストデータのミニバッチに対して、学習のときと同じ様に処理をしていきます。まず、test_loaderから画像データの束と正解ラベルの束を得ます。

for images, labels in test_loader:

続いて、

images = images.view(-1, 28*28)

y = mynet(images)

を実行して、画像の束に対して、出力の束yを得ます。ここで、出力yに対して、次の処理を行います。

pred_labels = y.max(dim=1)[1]

出力yは

[[画像0のクラス0の確率, 画像0のクラス1の確率,..., 画像0のクラス9の確率],

[画像1のクラス0の確率, 画像1のクラス1の確率,..., 画像1のクラス9の確率],

...

[画像99のクラス0の確率, 画像99のクラス1の確率,..., 画像99のクラス9の確率]]

という行列になっています。maxは行列の値の中での最大値を求める計算を行うのですが、dim=0を指定すると行列の各列の中での最大値を求めて、dim=1を指定すると行列の各行の中で最大値を求めます。ここでは、各行に各画像のクラスの確率のリストが入っているので、dim=1として、各画像毎に最大値を求めます。

ただし、ここでは最大確率ではなく、最大確率となるクラスを求めたいのですが、m

axは最大確率とその最大確率となるインデックスをペアで返してくれているので、最後に[1]をつけてあげると、最大確率となるインデックス、つまり、予測するクラスIDが手に入ります。

次にpred_labelsとlabelsを比較してクラスIDが正解と成っているものを数え上げます。forループで計算しても良いのですが、まとめて計算する良い方法がPyTorchに提供されています。

まず、(pred_labels == labels)を計算すると、正解ラベルと予測ラベルが一致しているところはTrue, 一致していないところはFalseとなるリストが返ってきます。例えば、

```
a = torch.tensor([1,2,3,4,5,6])
```

```
b = torch.tensor([2,3,3,4,1,6])
```

```
print(a == b)
```

に対して、

```
tensor([False, False, True, True, False, True])
```

といった結果が返ってきます。ブール型のリストに対して、sumを実行すると、Trueの数を数え上げてくれます。従って、

```
correct = correct + (pred_labels == labels).sum()
```

と書くことでpred_labelsとlabelsの中での一致数を計算して、correctに足し込むというを行っています。

最後に、correctの数を表示して、正解数を表示します。correctはテンソル型になっているので、item()をつけて整数型にします。続いて、totalを表示することで、全データ数を表示し、correct.item()/totalを計算することで正解率を求め、これを表示します。

実行結果

```
epoch: 0 loss: 195.63006672263145
epoch: 1 loss: 82.39563230797648
epoch: 2 loss: 54.87096862308681
epoch: 3 loss: 40.70517487358302
epoch: 4 loss: 31.048207819461823
epoch: 5 loss: 23.900350784882903
epoch: 6 loss: 19.02593113365583
epoch: 7 loss: 14.900030710035935
epoch: 8 loss: 12.13549662521109
epoch: 9 loss: 9.818495093728416
```

損失がだんだん減っていることがわかる

```
correct: 9798
total: 10000
accuracy: 0.9798
```

10000データ中、9798データの正解
(=97.98%の正解率)



- <http://yann.lecun.com/exdb/mnist/> の結果と比較してみよう

28

うまく実行できると上の図のように学習が行われます。エポックが進むにつれて、損失が減っていることがわかります。

下の図はテストデータの評価ですが、うまくいっていればだいたい97.2%~98.5%ぐらいの精度がでます。興味のある人は、<http://yann.lecun.com/exdb/mnist/> の結果と比較してみましょう。

まとめ

- PyTorchの使い方を学んだ
- 自動微分により、回帰モデルの学習ができることがわかった
- ニューラルネットワークによる画像認識を行った

今回はクラスを使わないPyTorchの書き方を勉強しましたが、クラスを使う書き方の方が一般的です。

次回、クラスを使ったPyTorchの使い方について学びます。



29

今回のまとめです。

今回はPyTorchの使い方を学びました。自動微分により、回帰モデルの学習ができることがわかりました。ニューラルネットワークを構築して、MNISTの画像分類ができることがわかりました。今回はクラスを使わないPyTorchの書き方を勉強しましたが、クラスを使う書き方の方が一般的です。次回はクラスを使ったPyTorchの使い方について勉強します。